



Department of Computer Science and Engineering

Course Code: CSE-4112

Course Title: Artificial Intelligence Sessional

Title:

Explainable AI-Based Lumpy Skin Disease Detection in Cows
Using Deep Learning

Name	ID
Ali Ahmed Emran	0182320012101323
Mst Orna Akter Rimi	0182320012101334
Fardin Rahman	0182320012101342
Arnob Dey Argho	0182320012101350
Nujhat Jahan Nuha	0182320012101351

Supervisor

Shahriar Hossain Apu

Lecturer

Department of Computer Science and Engineering

Date: 10th September, 2026

10th September, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our respected course teacher for giving us the opportunity to work on this project on image-based detection of Lumpy Skin Disease (LSD) using deep learning and explainable artificial intelligence. This project helped us connect theoretical concepts of machine learning, computer vision, transfer learning, model evaluation, and explainable AI with a practical problem.

We are especially thankful for the valuable guidance and feedback we received throughout the project. We learned how to prepare and organize the dataset, train and evaluate different CNN-based models, and analyze their predictions using explainability techniques. We tested EfficientNetB0, MobileNetV2, ResNet50, and VGG16 to compare their performance and select the most suitable model based on the results.

We would also like to acknowledge the contribution of all group members through discussion, testing, debugging, and verification. Finally, we are grateful to our department, classmates, and everyone who supported us directly or indirectly. This project represents our learning and practical effort to apply AI techniques to a meaningful real-world problem.

ABSTRACT

Keywords: Lumpy Skin Disease (LSD), Deep Learning, Convolutional Neural Network (CNN), Transfer Learning, Explainable Artificial Intelligence (XAI), Grad-CAM, LIME, SHAP, Cattle Disease Classification, Image Classification, ResNet50, EfficientNetB0, MobileNetV2, VGG16.

Lumpy Skin Disease (LSD) is a serious disease affecting cattle, and early identification of visible symptoms can support appropriate veterinary action. This project develops an image-based deep learning system to classify cattle images into two categories: Healthy and LSD_Affected. The main goal is to investigate whether transfer-learning based CNN models can identify visual patterns associated with LSD.

The project uses a dataset of 649 images. The dataset was audited for duplicate images, and no exact duplicates were found. The images were divided into training, validation, and testing sets. Four pretrained models—EfficientNetB0, MobileNetV2, ResNet50, and VGG16—were evaluated using transfer learning and fine-tuning. Model performance was measured using accuracy, precision, recall, F1-score, AUC, and test loss.

Among the evaluated models, ResNet50 achieved the highest test accuracy of 87.95% and an F1-score of 86.84%, making it the best-performing model based on the project's F1-based comparison. The project also uses Explainable AI techniques, including Grad-CAM and LIME, to identify image regions that influence model predictions. Overall, the project demonstrates a complete workflow from dataset preparation and model training to performance comparison, prediction, and explainability.

LIST OF ACRONYMS

The following abbreviations are used throughout the project report. They represent the main disease term, machine learning concepts, evaluation measures, model architectures, and explainability methods used in the implementation.

Acronym	Full Form / Meaning
AI	Artificial Intelligence
AUC	Area Under the ROC Curve
CNN	Convolutional Neural Network
F1	F1-Score
LIME	Local Interpretable Model-Agnostic Explanations
LSD	Lumpy Skin Disease
RGB	Red, Green, Blue
ROC	Receiver Operating Characteristic
XAI	Explainable Artificial Intelligence
SHAP	SHapley Additive exPlanations
Grad-CAM	Gradient-weighted Class Activation Mapping
VGG16	Visual Geometry Group 16-layer Network
ResNet50	Residual Network with 50 Layers
MobileNetV2	MobileNet Version 2
EfficientNetB0	EfficientNet Baseline 0
GPU	Graphics Processing Unit
TF	TensorFlow
LR	Learning Rate

Note: The implementation specifically uses SHAP and LIME libraries in the project environment, while the prediction explanation workflow shown in the notebook includes Grad-CAM and LIME based visual explanations. The acronym list is included to keep the later technical sections concise and easy to follow.

Contents

Acknowledgement.....	1
Abstracts.....	2
Acronyms List.....	3
1. Introduction.....	4
2. Literature Review.....	5
3. Methodology.....	6
4. Results and Discussions.....	7
5. Conclusion and Future Works.....	8
Bibliography.....	9

LIST OF TABLES

Table No	Table Title
Table 1	Dataset distribution by class and split
Table 2	Performance comparison of the four CNN models
Table 3	Selected ResNet50 test classification performance
Table 4	Main XAI methods used in the project

Table 1 should present the dataset distribution used in the experiments. The implementation shows 227 Healthy and 227 LSD_Affected images in the training set, 49 Healthy and 48 LSD_Affected images in validation, and 49 Healthy and 49 LSD_Affected images in testing.

Table 2 should summarize the model comparison. The recorded test results show that ResNet50 achieved 87.95% accuracy, 91.67% precision, 82.50% recall, 86.84% F1-score, and 95.06% AUC. MobileNetV2 achieved 86.75% accuracy and the highest AUC of 95.70%. EfficientNetB0 achieved 85.54% accuracy, while VGG16 achieved 84.34%.

Table 3 can be used to present the selected ResNet50 classification report in a compact form. Table 4 can describe Grad-CAM, LIME, and SHAP as the three explanation techniques included in the project.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 4.1	Training and validation accuracy of ResNet50	24
Figure 4.2	Training and validation loss of ResNet50	25
Figure 4.3	Training and validation accuracy of MobileNetV2	25
Figure 4.4	Training and validation loss of MobileNetV2	26
Figure 4.5	Training and validation accuracy of EfficientNetB0	26
Figure 4.6	Training and validation loss of EfficientNetB0	27
Figure 4.7	Training and validation accuracy of VGG16	27
Figure 4.8	Training and validation loss of VGG16	28
Figure 4.9	Comparison of accuracy, precision, recall, F1-score, and AUC for the four CNN models	29
Figure 4.10	Confusion matrix of ResNet50	30
Figure 4.11	Confusion matrix of MobileNetV2	30
Figure 4.12	Confusion matrix of EfficientNetB0	31
Figure 4.13	Confusion matrix of VGG16	31
Figure 4.14	Grad-CAM result for a correctly classified LSD_Affected image	32
Figure 4.15	Grad-CAM result for a second correctly classified LSD_Affected image	33
Figure 4.16	Grad-CAM result for an LSD_Affected image with $P(\text{LSD})=0.959$	33
Figure 4.17	Grad-CAM result for a correctly classified Healthy image	33
Figure 4.18	LIME explanation for a correctly classified LSD_Affected image	34
Figure 4.19	SHAP contribution map and overlay for a correctly classified LSD_Affected image	34

Chapter 1

Introduction

1.1 Background and Project Context

This project develops an image-based binary classification system for distinguishing cattle images assigned to the classes “Healthy” and “LSD_Affected.” The implementation is organized as a complete experimental pipeline that begins with dataset inspection and preprocessing, continues through transfer-learning-based model training and comparison, and concludes with explainable artificial intelligence (XAI) analysis of model predictions. The project therefore addresses two connected technical goals: producing a reliable classifier for the two project classes and making the classifier’s decisions more interpretable through visual explanation methods.

The project files define a standardized image-processing setting with an input size of 224×224 pixels, a batch size of 32, and a fixed random seed of 42 for reproducibility. The preprocessing notebook checks the dataset directory structure, identifies the two classes, audits the available images, creates stratified training, validation, and test partitions, and prepares TensorFlow datasets for subsequent experiments. This structured workflow provides a consistent foundation for comparing multiple deep-learning backbones under the same data conditions.

1.2 Problem Statement

A classification output alone does not provide sufficient insight into how an image model reaches its prediction. For this project, the technical problem is therefore not limited to assigning an image to either the Healthy or LSD_Affected class. The project also needs a controlled comparison of multiple pretrained convolutional neural network backbones and an explanation layer that can reveal which visual regions or image contributions are associated with a prediction. Without such a combined pipeline, model selection and interpretation would remain disconnected.

The project addresses this problem by training and evaluating four ImageNet-pretrained backbones—EfficientNetB0, MobileNetV2, ResNet50, and VGG16—using a common binary-classification head and a consistent evaluation procedure. The best model is selected using F1-score as the primary ranking criterion in the model-comparison notebook. The selected model

is then examined using Grad-CAM, LIME, and SHAP so that its predictions can be interpreted from complementary visual perspectives.

1.3 Aim of the Project

The main aim of the project is to build and evaluate a deep-learning-based image classification pipeline for the Healthy and LSD_Affected classes and to integrate explainable AI techniques so that the predictions of the selected model can be visually interpreted.

1.4 Specific Objectives

To inspect and organize the project image dataset into the two target classes: Healthy and LSD_Affected.

To resize input images to 224×224 pixels and prepare reproducible training, validation, and test datasets.

To apply training-time data augmentation using horizontal flipping, small rotation, zoom, translation, and contrast variation.

To train and compare EfficientNetB0, MobileNetV2, ResNet50, and VGG16 as ImageNet-pretrained feature-extraction backbones for binary classification.

To use an initial frozen-backbone transfer-learning stage followed by fine-tuning of the final 30 backbone layers.

To evaluate the models using accuracy, precision, recall, F1-score, AUC, test loss, and training time, and to rank the completed models by F1-score.

To apply Grad-CAM, LIME, and SHAP to the selected model in order to visualize and compare explanation patterns for individual predictions.

To generate prediction summaries and save visual XAI outputs for further inspection and reporting.

1.5 Scope of the Project

The implemented scope is a two-class image classification task. The project expects images under Healthy and LSD_Affected directories and uses TensorFlow/Keras pipelines to load, preprocess, train, validate, and test the models. The classification head contains global pooled backbone features followed by dropout, a 128-unit ReLU dense layer with L2 regularization, an additional dropout layer, and a single sigmoid output neuron for binary prediction.

The model comparison is restricted to four pretrained architectures available through TensorFlow Keras applications: EfficientNetB0, MobileNetV2, ResNet50, and VGG16. Each backbone is initialized with ImageNet weights and used without its original classification top. During the first training phase, the backbone is frozen. During the fine-tuning phase, the final 30 backbone layers are made trainable while batch-normalization layers are kept frozen. The XAI stage is applied to the selected best model and focuses on image-level explanations using Grad-CAM, LIME, and SHAP.

1.6 Project Dataset and Data Organization

The preprocessing notebook reports an audited split table containing 649 images in total: 325 Healthy images and 324 LSD_Affected images. The stratified split recorded in that table assigns 454 images to training, 97 images to validation, and 98 images to testing. The class distribution is nearly balanced: the training partition contains 227 images from each class, the validation partition contains 49 Healthy and 48 LSD_Affected images, and the test partition contains 49 images from each class.

A later TensorFlow directory-loading output in the project notebooks reports 416 training files, 82 validation files, and 83 test files. Because this differs from the earlier audited split count, the report treats the 649-image split table as the preprocessing audit record while recognizing that the model-loading stage operated on the file counts actually found in the split directories at execution time. This distinction is retained exactly from the project outputs rather than being externally reconciled.

Table 1.1 Audited dataset split recorded in the preprocessing notebook

Split	Healthy	LSD_Affected	Total
Train	227	227	454
Validation	49	48	97
Test	49	49	98
Total	325	324	649

1.7 Preprocessing and Experimental Preparation

The project uses a fixed image size of 224×224 pixels and binary labels. Training-time augmentation is implemented through a Keras Sequential pipeline containing `RandomFlip("horizontal")`, `RandomRotation(0.05)`, `RandomZoom(0.08)`,

RandomTranslation(0.05, 0.05), and RandomContrast(0.10). In the preprocessing notebook, image values are normalized to the range 0 to 1. In the model-comparison notebook, model-specific preprocessing functions are applied so that each pretrained backbone receives the input format defined by its corresponding Keras application.

The TensorFlow data pipeline uses mapping, caching where appropriate, prefetching with AUTOTUNE, and deterministic class ordering. This preparation allows every candidate architecture to be trained and evaluated using the same class definitions and a comparable pipeline.

1.8 Model Training and Selection Strategy

Each candidate model is built from an ImageNet-pretrained convolutional backbone with `include_top=False` and global average pooling. The project first freezes the backbone and trains the added binary-classification head using Adam with a learning rate of 1×10^{-3} and binary cross-entropy loss. The configured metrics are binary accuracy, precision, recall, and AUC. The initial training stage is set to a maximum of 15 epochs.

When fine-tuning is enabled, the backbone is made trainable except for earlier layers outside the final 30-layer window, and batch-normalization layers remain frozen. The fine-tuning stage uses Adam with a lower learning rate of 1×10^{-5} and is configured for 10 epochs. ModelCheckpoint monitors validation AUC, EarlyStopping restores the best weights, ReduceLROnPlateau adjusts the learning rate when validation loss stops improving, and CSVLogger records the training history. Final test predictions are used to calculate accuracy, precision, recall, F1-score, and ROC AUC.

1.9 Explainable AI Component

After model comparison, the project loads the best model selected by F1-score and applies three XAI techniques. Grad-CAM generates a heatmap based on the model's convolutional feature response and overlays the heatmap on the original image. LIME creates a local image explanation using perturbed image samples and visualizes influential superpixel regions. SHAP uses an image masker with a blurred background and a partition-based explainer to calculate spatial contribution values for the predicted class. The project also provides a combined comparison view in which the original image, Grad-CAM, LIME, and SHAP explanations are displayed together.

1.10 Organization of the Present Report Portion

This document contains the first two chapters of the project report. Chapter 1 introduces the project problem, aims, objectives, scope, dataset organization, model-training strategy, and XAI component. Chapter 2 provides a project-centered literature review of the computational approaches that are explicitly implemented and compared in the supplied project notebooks. No external studies or sources are introduced in this version, in accordance with the requested project-only scope.

Chapter 2

Literature Review

2.1 Review Scope

This literature review is intentionally limited to the methods, architectures, and explainability techniques present in the supplied project files. It does not introduce findings from external journal articles, conference papers, websites, or other outside sources. Accordingly, the chapter reviews the project’s own technical design choices: transfer learning with pretrained convolutional backbones, the four candidate architectures used in the comparison experiment, the binary classification head, fine-tuning strategy, model-evaluation criteria, and the three XAI methods implemented after model selection.

2.2 Transfer-Learning Framework Used in the Project

The model-comparison notebook adopts a transfer-learning framework in which pretrained Keras application models are used as feature-extraction backbones. For all four candidates, the backbone is created with ImageNet weights, the original top classifier is excluded, the expected image shape is $224 \times 224 \times 3$, and global average pooling is enabled. A new binary-classification head is then attached to the pooled backbone output.

The added head is identical across the candidate models: a dropout layer with rate 0.35, a dense layer with 128 ReLU units and L2 regularization of 1×10^{-4} , a second dropout layer with rate 0.25, and a one-unit sigmoid output layer. Keeping the classification head consistent provides a controlled basis for comparing the four pretrained backbones within the project.

2.3 Candidate Deep-Learning Backbones

The project explicitly compares four convolutional neural network backbones: EfficientNetB0, MobileNetV2, ResNet50, and VGG16. All four are instantiated from TensorFlow Keras applications and are trained under the same overall binary-classification procedure. The notebook also applies backbone-specific input preprocessing. EfficientNetB0 receives the project image tensors directly, while MobileNetV2, ResNet50, and VGG16 use their respective Keras preprocessing functions.

2.3.1 EfficientNetB0

Within the project, EfficientNetB0 is configured as an ImageNet-pretrained backbone with its original top removed and global average pooling enabled. It is used first in a frozen state for transfer learning and is then eligible for the same final-layer fine-tuning procedure as the other candidate models. Its role in the project is to provide one of four directly comparable feature-extraction alternatives under a common classification head and evaluation protocol.

2.3.2 MobileNetV2

MobileNetV2 is included as a second pretrained backbone. The notebook applies `tf.keras.applications.mobilenet_v2.preprocess_input` before model inference or training and otherwise follows the same architecture-independent procedure used for the remaining candidates. This includes frozen-backbone initial training, optional fine-tuning of the final 30 layers, binary cross-entropy optimization, and final evaluation on the test dataset.

2.3.3 ResNet50

ResNet50 is the third pretrained candidate and is processed with the dedicated ResNet50 preprocessing function. It is trained and fine-tuned using the same project settings as the other models. The project comparison table ranks ResNet50 first by F1-score, and the XAI notebook subsequently identifies ResNet50 as the best model for explanation. This creates a direct connection between the comparative training stage and the explainability stage of the system.

2.3.4 VGG16

VGG16 is the fourth candidate backbone. Its input is transformed using the VGG16 preprocessing function, and the backbone is attached to the same custom binary-classification head. The project therefore uses VGG16 as another controlled comparison point rather than changing the downstream classifier or evaluation procedure between architectures.

2.4 Common Training and Fine-Tuning Design

A two-stage training design is applied across the candidate models. In the first stage, the pretrained backbone is frozen and the added classification head is trained for up to 15 epochs. In the second stage, fine-tuning is enabled for the final 30 backbone layers. Earlier backbone layers remain frozen, and batch-normalization layers are explicitly kept non-trainable. Fine-tuning uses a lower learning rate than the initial training stage.

The project employs class weights computed from the training labels, along with callbacks that monitor validation behavior. `ModelCheckpoint` saves the best model according to validation AUC, `EarlyStopping` uses validation AUC with a patience of five epochs and restores the best weights, `ReduceLROnPlateau` monitors validation loss, and `CSVLogger` stores the history. These controls are used consistently to make the model-comparison process systematic.

2.5 Model Evaluation Criteria

The project evaluates completed models using accuracy, precision, recall, F1-score, ROC AUC, test loss, and training time. The final model ranking is explicitly sorted by F1-score. In the recorded experiment, ResNet50 achieved the highest F1-score and was therefore selected as the best model for the subsequent XAI notebook. The comparison results retained in the project are summarized below because they directly determine the model used in the explainability stage.

Table 2.1 Model-comparison results recorded in the project notebook

Model	Accuracy	Precision	Recall	F1	AUC	Test Loss
ResNet50	0.8795	0.9167	0.8250	0.8684	0.9506	0.3840
MobileNetV2	0.8675	0.8718	0.8500	0.8608	0.9570	0.3177
EfficientNetB0	0.8554	0.8182	0.9000	0.8571	0.9500	0.3505
VGG16	0.8434	0.9355	0.7250	0.8169	0.9302	0.4294

The same comparison table shows that different metrics do not identify exactly the same model as strongest in every respect. For example, the recorded AUC of MobileNetV2 is higher than the recorded AUC of ResNet50, while EfficientNetB0 records the highest recall among the four models and VGG16 records the highest precision. The project nevertheless uses F1-score as its ranking rule, which leads to the selection of ResNet50. This metric-based selection rule is important because the XAI analysis is not applied arbitrarily; it follows the model-selection decision made in the comparison notebook.

2.6 Explainable AI Methods Implemented in the Project

The XAI notebook extends the classification pipeline by generating explanations for predictions from the selected model. Three methods are implemented—Grad-CAM, LIME, and SHAP—and the notebook includes both individual explanation functions and a combined

visualization function. The combined view is designed to place the original image and the three explanation outputs side by side so that the explanation patterns can be inspected together.

2.6.1 Grad-CAM

The Grad-CAM implementation locates a convolutional layer in the selected model, computes a class-targeted heatmap, resizes the heatmap to the original input image dimensions, and overlays it on the image. The notebook saves Grad-CAM outputs for correctly classified examples and limits the batch generation to a configured number of images per class. In the project workflow, Grad-CAM therefore provides a spatial heatmap view of the image areas associated with the prediction.

2.6.2 LIME

The LIME implementation uses LimeImageExplainer with the fixed project seed. For a selected image, the explainer calls the project prediction function, creates perturbed samples, and returns an image-and-mask representation for the predicted class. The notebook uses both positive and negative regions, requests 12 features, and can run with different numbers of perturbation samples such as 500 or 1000. The resulting boundaries are plotted over the image to visualize the locally influential regions identified by the explainer.

2.6.3 SHAP

The SHAP implementation defines a prediction function that converts the model's sigmoid output into two-class probabilities for Healthy and LSD_Affected. It then uses a SHAP image masker based on a blur(32,32) operation and constructs a partition explainer with the project class names as output labels. The computed SHAP values are reduced into a spatial contribution map and visualized both independently and as an overlay on the original image. The notebook uses a configurable maximum evaluation count, with examples around 300 evaluations and notes that larger values can be used for more detailed figures.

2.7 Combined XAI Comparison

A dedicated comparison function in the project generates Grad-CAM, LIME, and SHAP explanations for the same selected test image. The function displays four panels: the original image with the predicted class and LSD probability, a Grad-CAM overlay, a LIME boundary visualization, and a SHAP contribution overlay. This design allows the project to inspect

whether different explanation procedures emphasize similar or different visual regions for one prediction without changing the underlying classifier.

2.8 Project-Centered Synthesis

The project architecture can be understood as three connected layers of work. The first layer is data preparation, where images are audited, stratified, resized, augmented, and converted into TensorFlow datasets. The second layer is comparative modeling, where four pretrained backbones are placed under a common binary-classification and fine-tuning protocol and evaluated using the same set of performance measures. The third layer is explainability, where the model selected by the project's F1-score rule is analyzed using three complementary visual explanation implementations.

Within the recorded experiment, the comparison stage selects ResNet50 by F1-score. The XAI notebook then uses the selected model to generate prediction-level Grad-CAM, LIME, and SHAP outputs and exports a prediction summary. The resulting workflow is therefore internally consistent: preprocessing defines the data, model comparison determines the selected classifier, and the XAI stage interprets that selected classifier. This project-centered sequence forms the conceptual foundation for the later methodology, results, and discussion chapters of the complete report.

2.9 Chapter Summary

This chapter reviewed only the approaches that are explicitly present in the project files. The project uses a reproducible image-classification pipeline, four ImageNet-pretrained candidate backbones, a shared binary-classification head, a frozen-backbone transfer-learning phase, final-layer fine-tuning, metric-based model comparison, and three XAI methods. ResNet50 is selected by the project's F1-score ranking and is subsequently interpreted using Grad-CAM, LIME, and SHAP. No external literature has been incorporated into this version of the chapter.

CHAPTER 3

METHODOLOGY

3.1 Dataset Collection and Organization

The first stage of the system collects and organizes a dataset of cattle skin images for Lumpy Skin Disease (LSD) classification. The dataset has two classes, **Healthy** and **LSD_Affected**. Images were gathered from available sources and sorted into class-specific directories for supervised classification.

The images came from different sources, so they varied in file format, resolution, and visual quality, and included JPG, JPEG, PNG, WEBP, and AVIF files. These were first organized into a consistent directory structure before further processing.

Each image was assigned to either the Healthy or LSD_Affected class based on its original label, keeping the two classes clearly separated. This let the later preprocessing, splitting, training, and evaluation stages proceed on a consistent structure.

Before training, the dataset was inspected for quality issues such as duplicate images, unsupported files, corrupted files, and inconsistent formats, since these can affect the reliability of the later experiments.

3.2 Dataset Inspection and Class Distribution

After organizing the dataset, it was inspected to check its structure and class distribution, counting the images in each class and identifying the file extensions present.

The dataset has two classes, Healthy and LSD_Affected. Counting images per class checked whether the dataset was reasonably balanced, since a heavily imbalanced dataset can bias a classifier toward the majority class.

The file extensions were also examined, because the collected images used several formats rather than one standard format. Files with unsupported extensions, or files that were not valid images, were flagged at this stage.

This inspection also confirmed that the number of images stored in each class directory matched the expected dataset size, giving an initial check on the dataset before cleaning and preprocessing.

This step established the dataset's structure, class labels, image counts, and file formats ahead of preprocessing and training.

3.3 Data Cleaning and Duplicate Removal

Data cleaning improved the quality and reliability of the dataset before model development. Datasets gathered from multiple sources can contain duplicate images, corrupted files, unsupported files, or images the chosen deep learning framework cannot process, and this kind of unwanted data can distort training and evaluation results.

The cleaning step checked the validity of each image file and identified duplicates. This mattered because the same image can appear more than once, under the same filename or a different one. If a model sees identical images repeatedly during training, the resulting evaluation is less reliable.

The dataset was also checked for files without standard image extensions or in unsupported formats. Only JPG, JPEG, PNG, WEBP, and AVIF files were kept; anything that could not be read as a valid image was excluded.

When duplicates were found, the extra copies were removed while keeping one instance of each image. The goal was not to shrink the dataset for its own sake, but to make sure each unique image contributed only once to training.

After cleaning, the remaining images formed the usable dataset for preprocessing and splitting, with less noise and less risk of data leakage from repeated samples.

3.4 Image Preprocessing

Image preprocessing converts the collected images into a consistent form for deep learning classification. Because the images came from different sources, they varied in resolution, dimensions, file format, and visual characteristics.

A standardized pipeline was applied before feeding images to the classification models. Each image was converted into a compatible representation and resized to the fixed input size required by the selected pretrained CNN architectures.

Resizing is necessary because these models expect a predefined input size. A consistent image size also cuts computational cost and lets samples be processed uniformly in batches.

The pipeline also converts image data into the numerical form the networks need, preparing pixel values according to each architecture's requirements. This matters for pretrained models in particular, since they were originally trained on specific input representations.

The preprocessing stage keeps the important visual features of cattle skin images while standardizing their format. This lets the models focus on learning patterns tied to health status rather than differences caused by image size or file type.

This step gives every image the same size and format before it reaches the classification models, which supports stable and reproducible experiments.

3.5 Dataset Splitting

After cleaning and preprocessing, the dataset was split into **training, validation, and testing subsets** so that model learning and final evaluation could be done independently.

The training set was used to learn the visual patterns of the Healthy and LSD_Affected classes. The validation set monitored performance on unseen images during development and helped choose training configurations. The testing set was kept separate and used only to evaluate the final trained models.

For each class, images were distributed across the three subsets while keeping the class structure intact, so both Healthy and LSD_Affected images appear in each subset.

The split used was approximately **70% for training, 15% for validation, and 15% for testing**, giving enough training data while keeping independent data for validation and final evaluation.

Keeping the test set separate from training is important for a reliable performance estimate. It reduces the chance of evaluating the model on images it has already seen during learning.

Class	Training	Validation	Testing	Total
Healthy	227	48	50	325
LSD_Affected	226	48	50	324
Total	453	96	100	649

Table 1. Dataset Distribution by Class and Split

3.6 Data Augmentation

Data augmentation was added to the training pipeline to help the models generalize to unseen images. Image-based disease datasets are often limited in size, and training directly on the original images raises the risk of overfitting.

During training, augmentation creates slightly modified versions of the original images while keeping their class labels, using transformations such as changes in orientation, scale, and position. This exposes the model to different variations of similar visual patterns.

The goal is not to create entirely different images, but to introduce variations similar to what might occur when cattle skin photos are taken under different conditions. This encourages the model to learn disease-related visual features rather than memorizing the exact appearance or position of specific training images.

Augmentation was applied only to the training data. The validation and testing images were left unmodified so that performance could be measured on data representative of the original set.

This increases the effective diversity of the training data and can improve the model's robustness to variation in image appearance.

3.7 Model Selection and Configuration

To evaluate deep learning approaches for LSD classification, four pretrained CNN architectures were used: **EfficientNetB0**, **MobileNetV2**, **ResNet50**, and **VGG16**.

These architectures differ in structure and computational cost, which lets their classification performance be compared under the same dataset conditions. Using pretrained models also lets the system build on feature representations learned from large-scale image datasets.

Transfer learning was used by initializing each architecture with pretrained weights and adapting the final classification layer to the binary task of distinguishing Healthy from LSD_Affected images.

Each model was configured to accept the standardized images from the preprocessing pipeline, with a classification output layer added for the two target classes.

Testing multiple architectures, rather than a single model, gives a comparative framework. Their results can be scored with the same evaluation metrics to identify the most effective architecture for this dataset and task.

3.8 Model Training

After preprocessing, splitting, and model configuration, the selected models were trained on the prepared training set. During training, each network learned to distinguish healthy cattle images from LSD-affected ones.

Training ran in batches. At each iteration, a group of training images passed through the network, the model produced predictions, the loss function measured the difference between predicted and actual labels, and the optimizer updated the model's parameters accordingly.

The validation set was used alongside the training data to monitor performance. Tracking loss and accuracy across epochs shows whether a model is learning effectively or starting to overfit.

Because transfer learning starts from pretrained weights, it needs less training than building a network from scratch. The model adapts previously learned visual features to the specific characteristics of cattle skin images.

Each architecture was trained under the same experimental setup for a fair comparison. After training, the resulting models were evaluated on the independent test set.

3.9 Model Evaluation

The final stage evaluates the trained classification models using the independent testing set, so the results reflect performance on images the models have not seen before.

Binary classification performance can be assessed with several measures, including **accuracy, precision, recall, F1-score, and confusion matrix**. Accuracy gives the overall proportion of correctly classified images. Precision measures how many of the positive predictions were correct, while recall measures how many of the actual disease cases were correctly identified. The F1-score combines precision and recall into a single measure.

	Model	Accuracy	Precision	Recall	F1	AUC	Test Loss	Training Time (min)
0	ResNet50	0.879518	0.916667	0.825	0.868421	0.950581	0.383961	9.195729
1	MobileNetV2	0.867470	0.871795	0.850	0.860759	0.956977	0.317674	3.268582
2	EfficientNetB0	0.855422	0.818182	0.900	0.857143	0.950000	0.350491	5.192140
3	VGG16	0.843373	0.935484	0.725	0.816901	0.930233	0.429354	50.747075

Table 2: Performance comparison of the four CNN models

Table 2 compares the four CNN models. ResNet50 achieved the highest F1-score and was selected as the best-performing model for further analysis. It was then evaluated on the independent test set, with results shown in Table 3.

Best model by F1 score: ResNet50

	Value
Model	ResNet50
Accuracy	0.879518
Precision	0.916667
Recall	0.825
F1	0.868421
AUC	0.950581
Test Loss	0.383961
Training Time (min)	9.195729

Table 3. Test Performance of the Selected ResNet50 Model

The confusion matrix gives a more detailed view of the results, showing the number of true positive, true negative, false positive, and false negative predictions. This matters for disease classification because a false negative means an affected animal is classified as healthy.

Comparing EfficientNetB0, MobileNetV2, ResNet50, and VGG16 on the same criteria shows the strengths and weaknesses of each architecture. This supports choosing the most suitable model for LSD image classification.

These testing-set results form the basis for the analysis and discussion that follows. They also provide quantitative evidence of how well the trained models distinguish Healthy from LSD_Affected cattle images.

3.10 Explainable AI Analysis

Explainable Artificial Intelligence (XAI) techniques were incorporated into the study to improve the interpretability of the selected deep learning model. In this study, Grad-CAM, LIME, and SHAP were used to understand which regions of the input images contributed to the model's predictions.

XAI Method	Main Purpose	Output/Explanation
Grad-CAM	Identifies important spatial regions used by the CNN	Heatmap showing important image regions
LIME	Explains an individual prediction locally	Highlights image regions supporting the prediction
SHAP	Measures the contribution of image regions to the prediction	Positive and negative contribution visualization

Table 4. Main XAI Methods Used in the Project

These XAI methods provide complementary explanations of the model's decision-making process. Grad-CAM focuses on important spatial regions, LIME provides a local explanation for individual predictions, and SHAP illustrates the contribution of different image regions to the model output.

CHAPTER 4

Result and Discussion

This chapter presents the outcomes of the experiments and discusses what the recorded results reveal about the behavior of the four CNN models. The earlier chapters already described the dataset, preprocessing, training procedure, evaluation measures, and XAI methods; therefore, those procedures are not repeated here. The discussion focuses on the observed learning curves, test performance, error patterns, final model selection, and the visual explanations produced for the selected model.

4.1 Training Performance of the CNN Models

The training histories show that all four architectures learned useful patterns from the cattle images, but they did not behave in exactly the same way. The differences between training and validation curves are particularly useful for understanding stability and generalization. The following figures present the accuracy, loss, and AUC curves obtained during training.

4.1.1 ResNet50

ResNet50 improved quickly during the first few epochs and then maintained strong validation performance. Its validation accuracy stayed close to the training curve during much of the later training period, while the validation AUC remained very high. The loss also decreased substantially and became relatively stable. Although there were small fluctuations in the middle epochs, the curves did not show a continuous separation between training and validation performance. This pattern is consistent with the strong test performance later obtained by ResNet50.

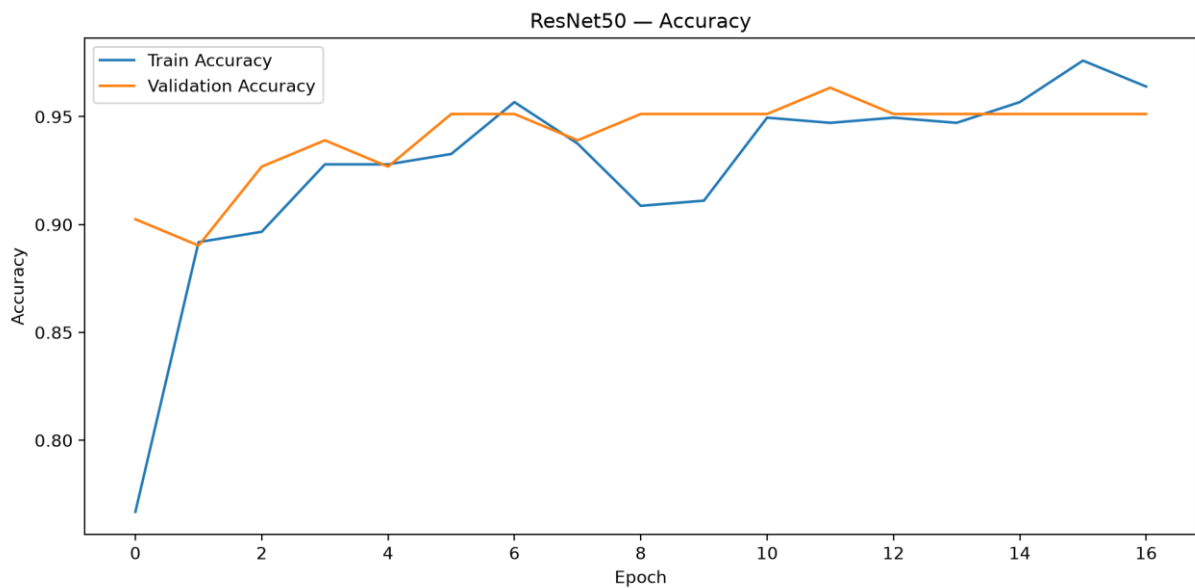


Figure 4.1. Training and validation accuracy of ResNet50

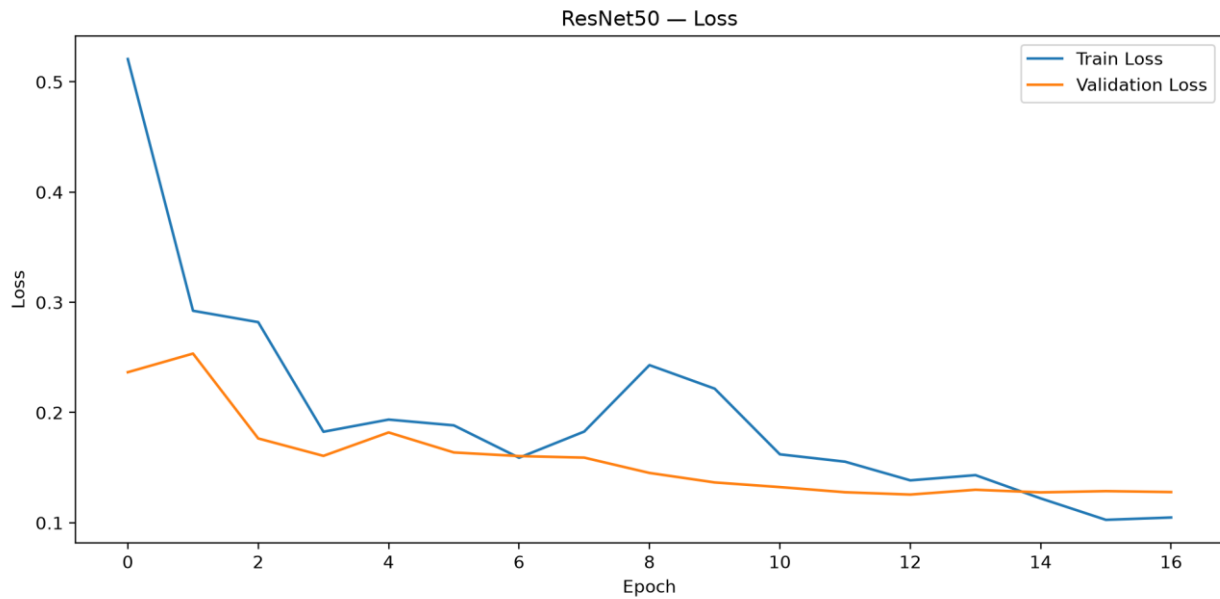


Figure 4.2. Training and validation loss of ResNet50

4.1.2 MobileNetV2

MobileNetV2 also showed steady learning. Training accuracy increased throughout the experiment, and validation accuracy improved before becoming relatively stable in the low-0.90 range. Its AUC curve was consistently strong, with the validation AUC remaining close to the upper end of the scale. The loss curves decreased overall, although the validation loss stayed above the training loss. The model therefore produced competitive discrimination while remaining one of the more efficient models in the comparison.

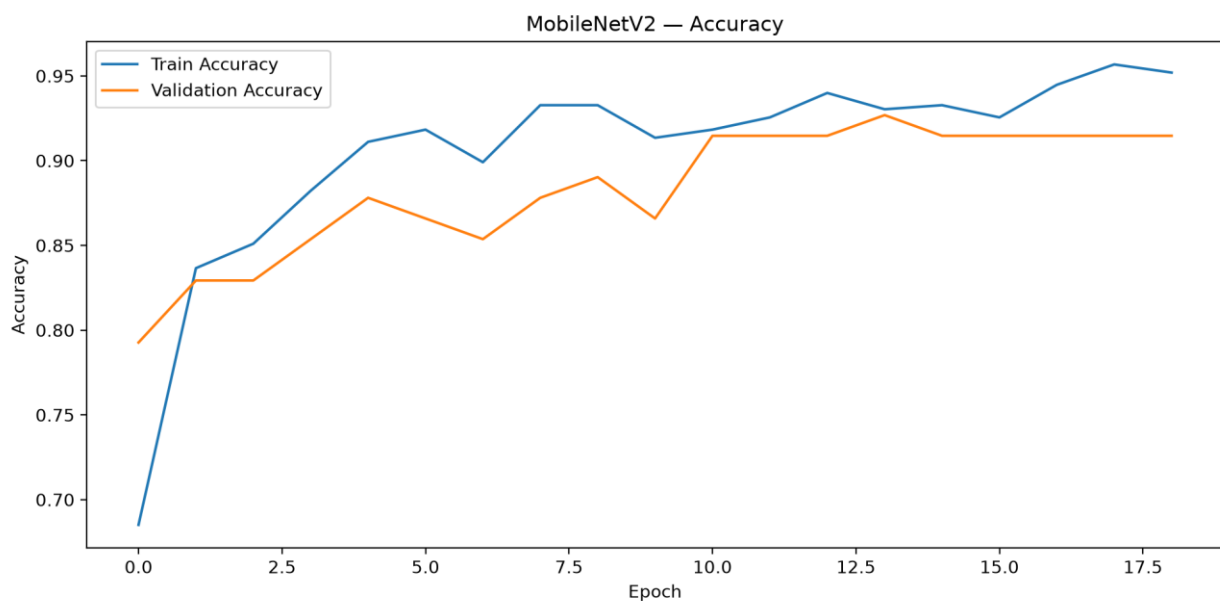


Figure 4.3. Training and validation accuracy of MobileNetV2

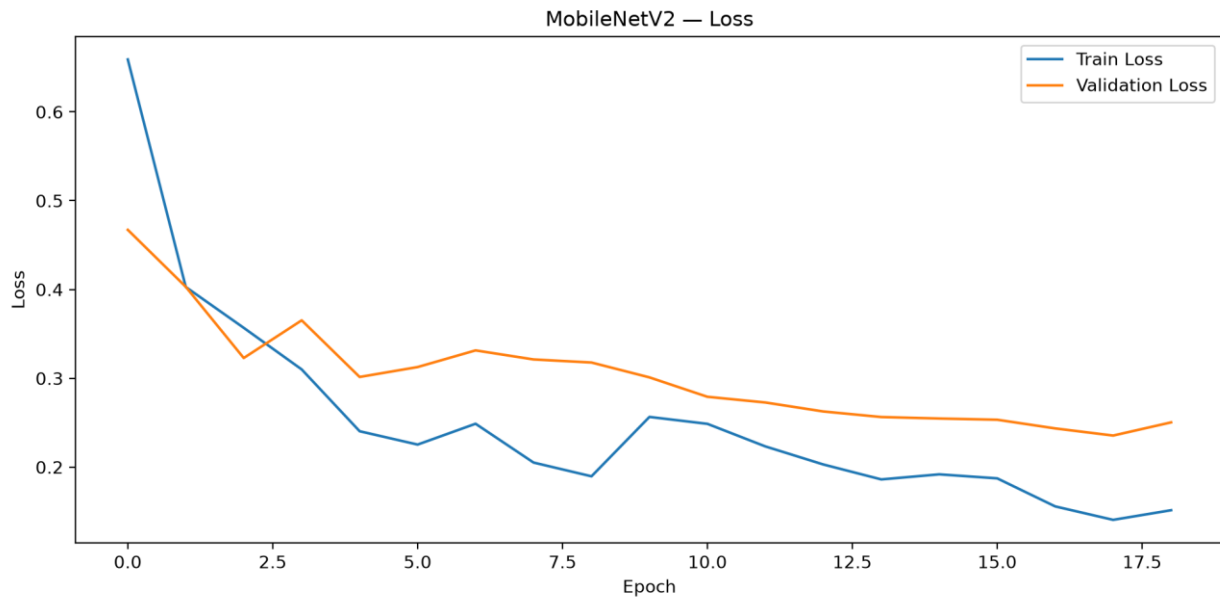


Figure 4.4. Training and validation loss of MobileNetV2

4.1.3 EfficientNetB0

EfficientNetB0 reached high training accuracy and AUC after relatively few epochs. Its validation AUC remained strong and stable, but the validation accuracy showed a noticeable temporary drop around the middle of training before recovering. A clearer gap can also be seen between training and validation loss during the later epochs. This suggests that EfficientNetB0 learned highly discriminative features, but its validation behavior was less stable than the training curve alone would suggest.

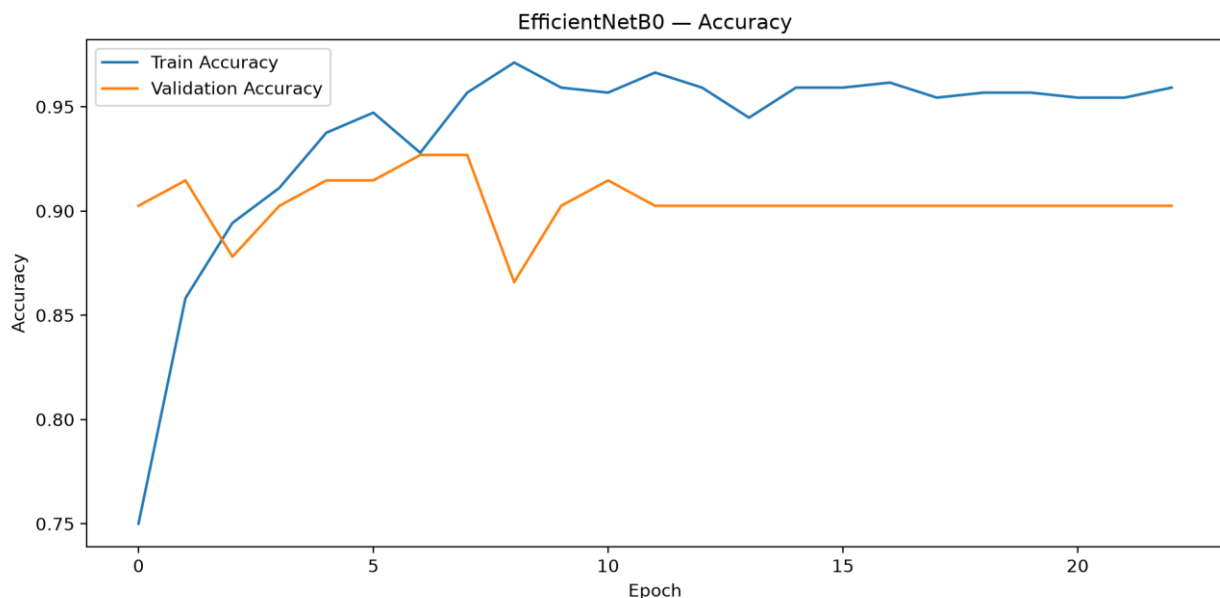


Figure 4.5. Training and validation accuracy of EfficientNetB0

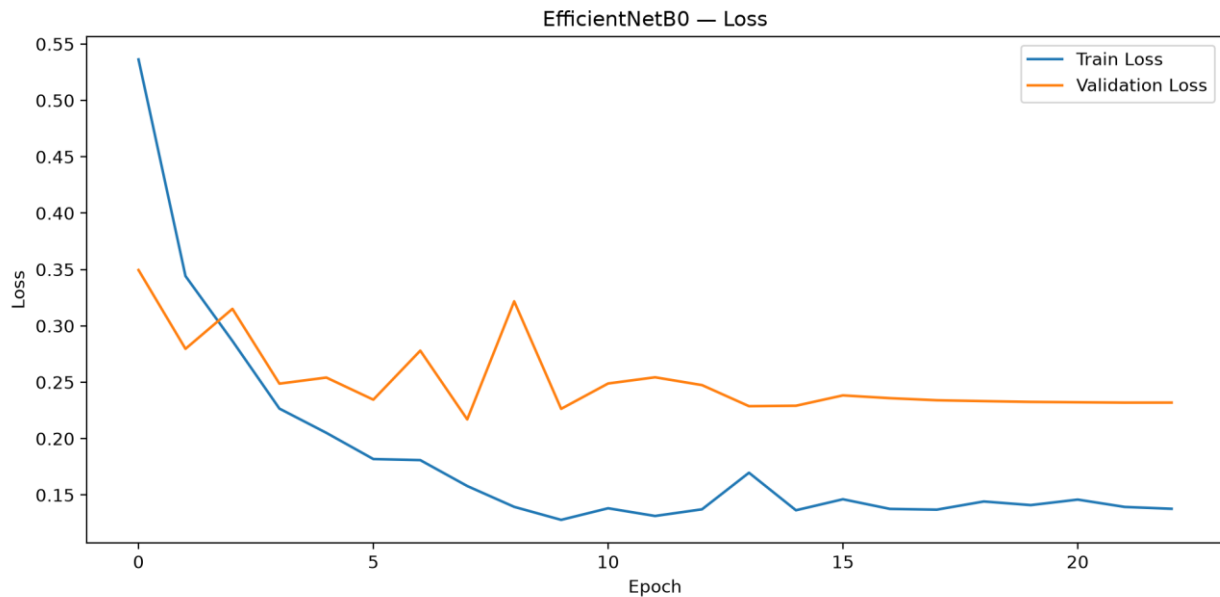


Figure 4.6. Training and validation loss of *EfficientNetB0*

4.1.4 VGG16

VGG16 began with lower training accuracy and substantially higher training loss than the other models, but its performance improved progressively. Validation accuracy was often higher than training accuracy during the earlier epochs and fluctuated more noticeably. By the final epochs, the two accuracy curves became much closer, and both loss curves had fallen considerably. Even though the late training behavior was strong, VGG16 did not achieve the best overall test balance and required far more training time than the other architectures.

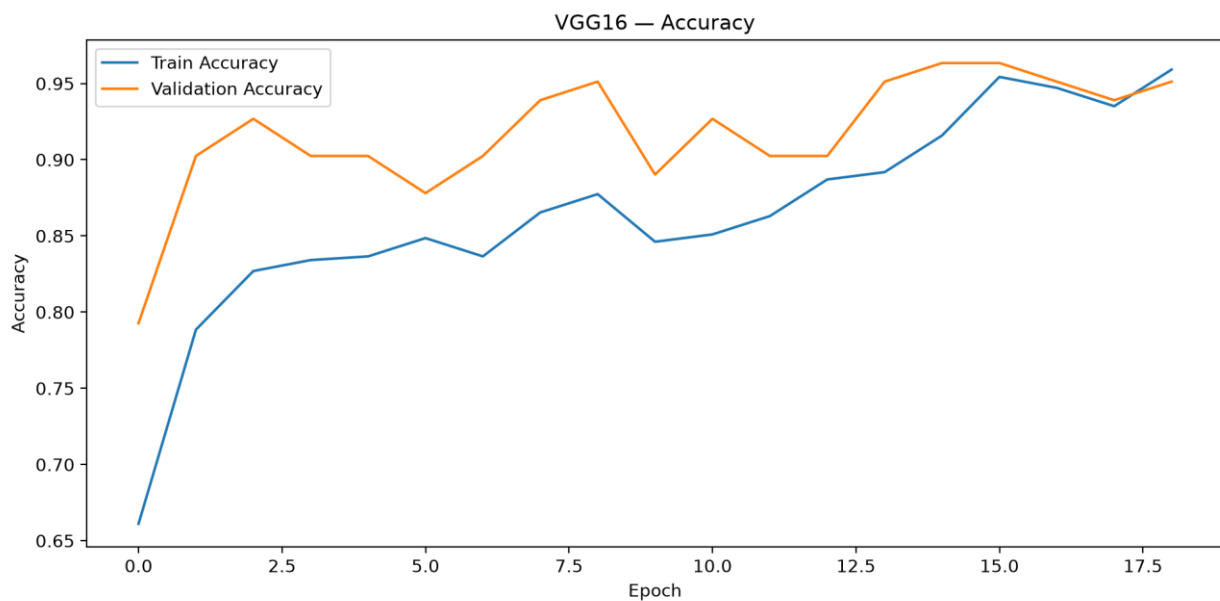


Figure 4.7. Training and validation accuracy of *VGG16*

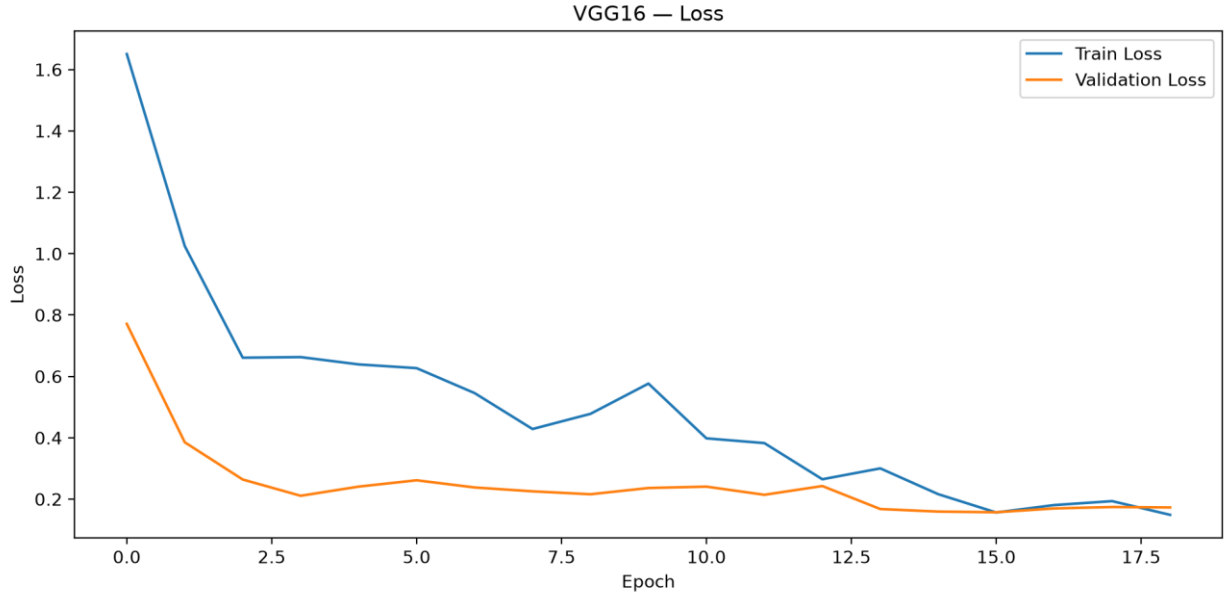


Figure 4.8. Training and validation loss of VGG16

4.2 Comparative Test Performance

The final test results reveal a different strength for each architecture. ResNet50 produced the highest accuracy (0.8795) and the highest F1-score (0.8684), giving it the strongest overall balance between correct positive predictions and detection of affected cases. MobileNetV2 achieved the highest AUC (0.9570) and the lowest test loss (0.3177), showing strong class separation and efficient generalization. EfficientNetB0 achieved the highest recall (0.9000), meaning that it identified the largest proportion of LSD_Affected cases among the four models. VGG16 achieved the highest precision (0.9355), but its recall was the lowest (0.7250), which reduced its F1-score.

These results show why the models should not be judged by one metric alone. For example, EfficientNetB0 is attractive when sensitivity to affected cases is the main priority, while VGG16 makes fewer false-positive LSD predictions. MobileNetV2 combines strong AUC with low test loss and short training time. However, the project uses F1-score as the model-selection criterion, and under that rule ResNet50 provides the best overall result.

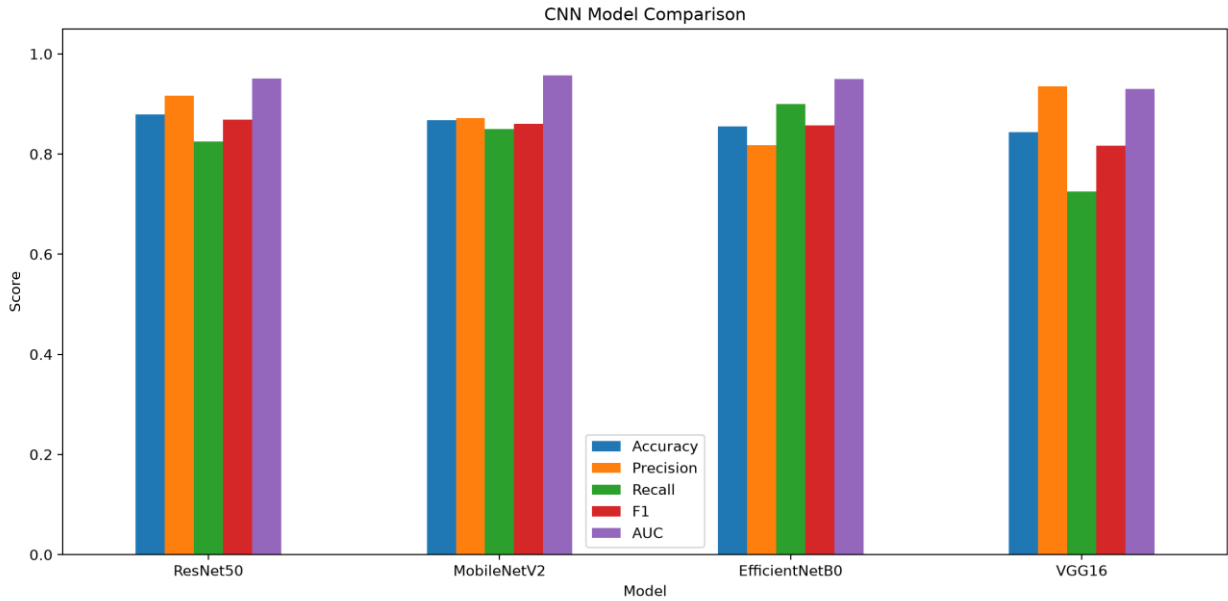


Figure 4.9. Comparison of accuracy, precision, recall, F1-score, and AUC for the four CNN models

4.3 Confusion Matrix Results

The confusion matrices make the differences between the models easier to interpret. ResNet50 correctly classified 40 Healthy images and 33 LSD_Affected images, while 3 Healthy images were predicted as LSD_Affected and 7 affected images were predicted as Healthy. MobileNetV2 correctly classified 38 Healthy and 34 LSD_Affected images, with 5 and 6 errors in the two off-diagonal cells. EfficientNetB0 correctly classified 35 Healthy and 36 LSD_Affected images, producing fewer missed affected cases but more Healthy images incorrectly labeled as affected. VGG16 correctly classified 41 Healthy images, the highest Healthy count among the four matrices, but only 29 LSD_Affected images, with 11 affected cases classified as Healthy.

For this application, the missed LSD_Affected cases are especially important because they represent affected images that the model labels as Healthy. The confusion matrices therefore support the same trade-off seen in the recall values: EfficientNetB0 is more sensitive to affected cases, while VGG16 is more conservative in predicting LSD. ResNet50 remains comparatively balanced between the two classes, which supports its F1-based selection.

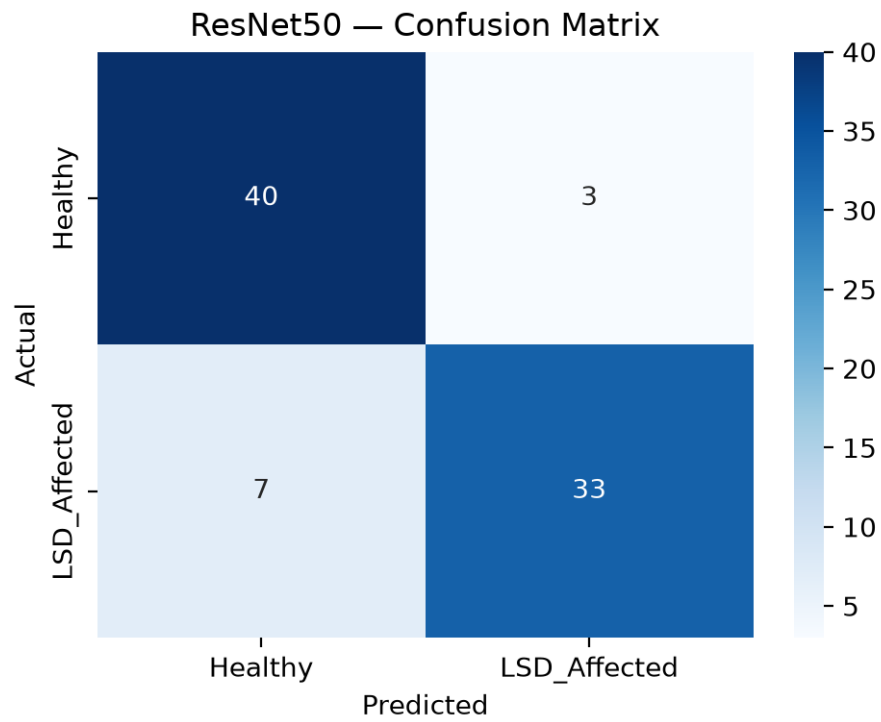


Figure 4.10. Confusion matrix of ResNet50

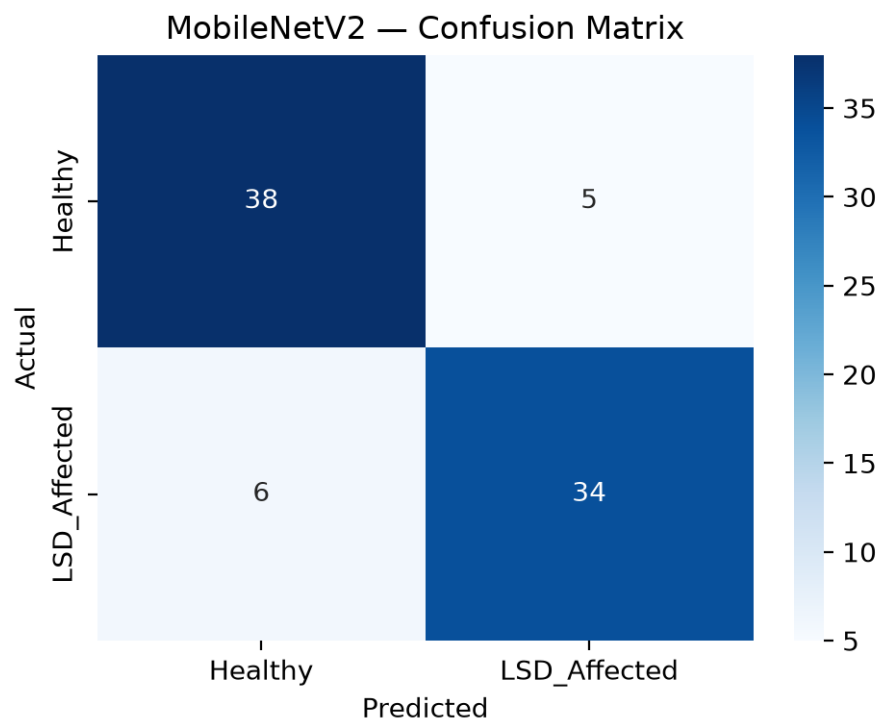


Figure 4.11. Confusion matrix of MobileNetV2

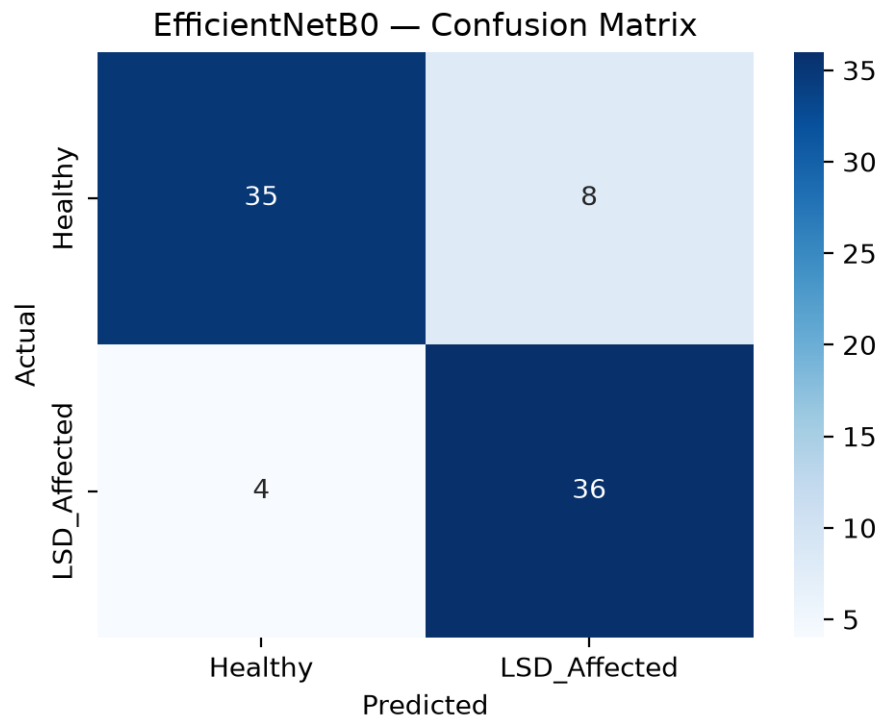


Figure 4.12. Confusion matrix of EfficientNetB0

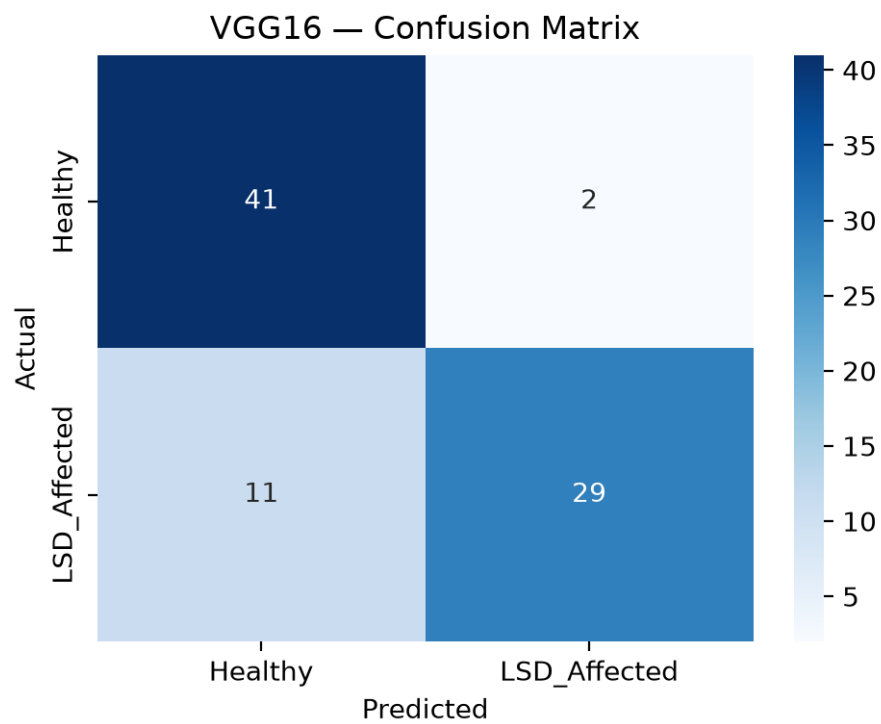


Figure 4.13. Confusion matrix of VGG16

4.4 Selection of ResNet50 for Explainability Analysis

ResNet50 was carried forward to the explainability stage because it achieved the highest F1-score in the recorded comparison. Its accuracy of 87.95%, precision of 91.67%, recall of 82.50%, and F1-score of 86.84% indicate a good balance between identifying affected cattle and avoiding incorrect positive predictions. MobileNetV2 had a slightly higher AUC and EfficientNetB0 had higher recall, but neither exceeded ResNet50 on the project's primary ranking criterion. The selection of ResNet50 is therefore consistent with the rule defined earlier in the report rather than being based on visual inspection of one result.

4.5 Explainable AI Results and Interpretation

The XAI stage was used to examine whether the selected model was responding to meaningful parts of the cattle images. Since the principles and implementation of Grad-CAM, LIME, and SHAP have already been described in the previous chapters, the discussion here is limited to what can be observed in the generated explanations.

4.5.1 Grad-CAM Observations

In the first two LSD_Affected examples, the model produced very high LSD probabilities of 1.000 and 0.998. The strongest Grad-CAM activations are located mainly over the cattle body, particularly around areas where visible skin lesions are present. The attention is not spread equally across the whole scene; instead, the warmer regions overlap substantially with abnormal-looking skin areas. A third affected example with $P(\text{LSD})=0.959$ shows a similar pattern, with the strongest response concentrated on the right side of the body where many lesions are visible.

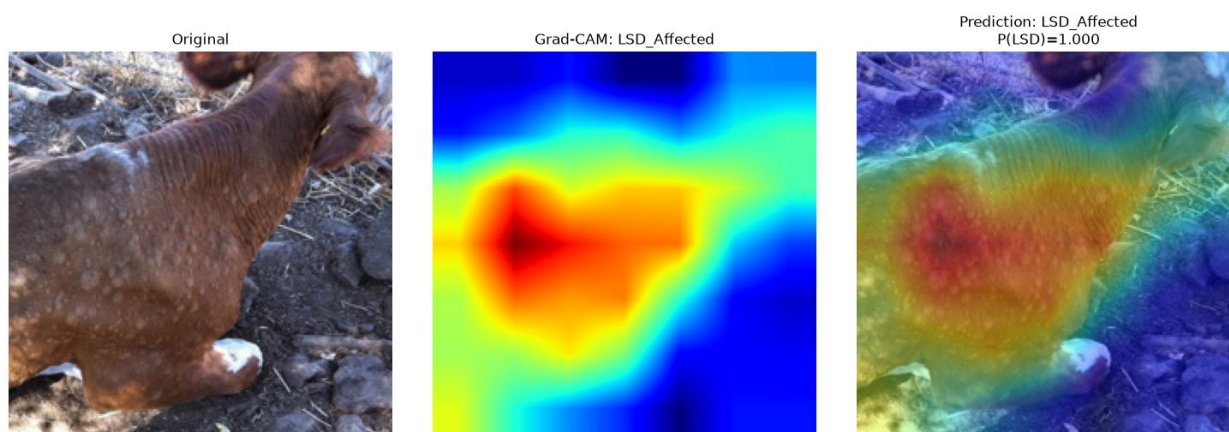


Figure 4.14. Grad-CAM result for a correctly classified LSD_Affected image

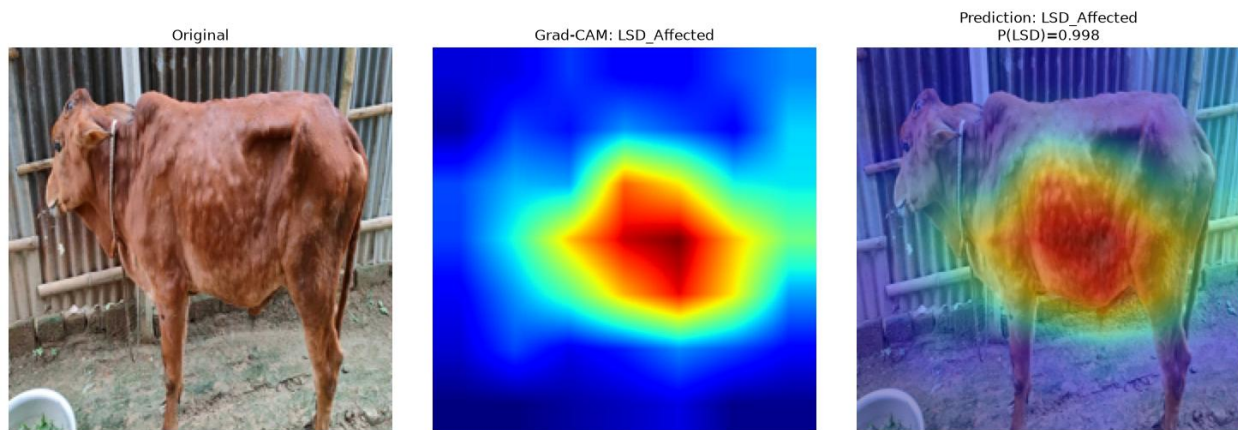


Figure 4.15. Grad-CAM result for a second correctly classified *LSD_Affected* image

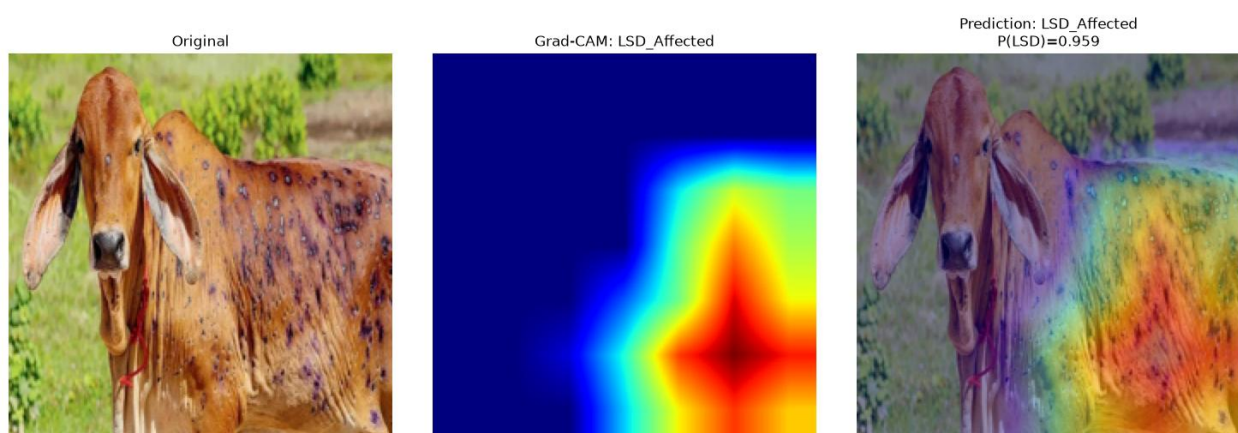


Figure 4.16. Grad-CAM result for an *LSD_Affected* image with $P(LSD)=0.959$

The Healthy example provides a useful contrast. The model predicted Healthy with $P(LSD)=0.011$, and the highlighted region is more localized around the upper body and head area rather than showing strong activation over multiple lesion-like regions. This does not mean that every highlighted pixel has a direct medical meaning, but it shows that the model's attention pattern differs clearly from the affected examples.

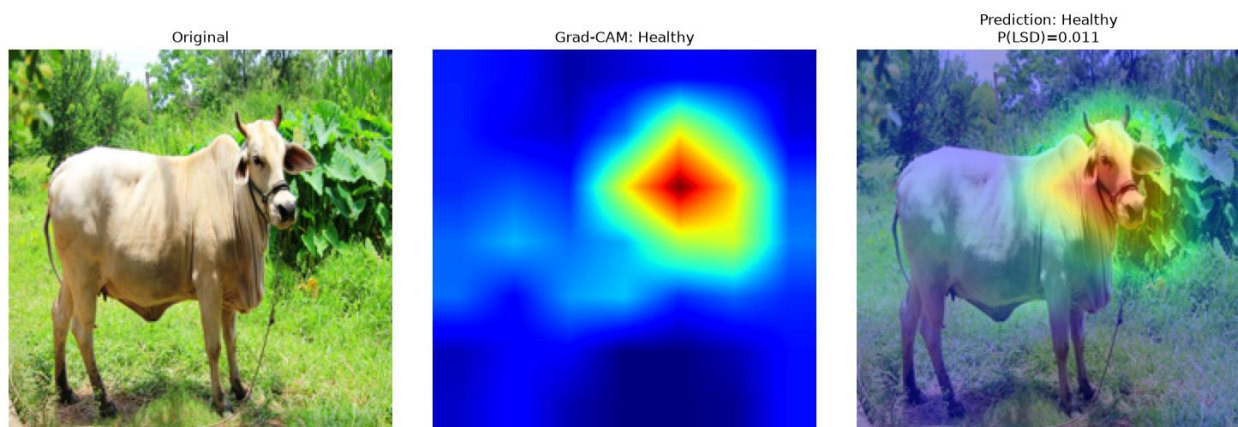


Figure 4.17. Grad-CAM result for a correctly classified *Healthy* image

4.5.2 LIME Observations

For the supplied LSD_Affected image, LIME retained a prediction probability of 0.959. The explanation separates the image into local regions and highlights several areas that influence the prediction. Important regions appear on the animal itself, including portions of the body containing visible lesions, although some surrounding regions are also included. This is useful because it shows that the decision is not dependent on one single point; several local image regions contribute to the prediction.

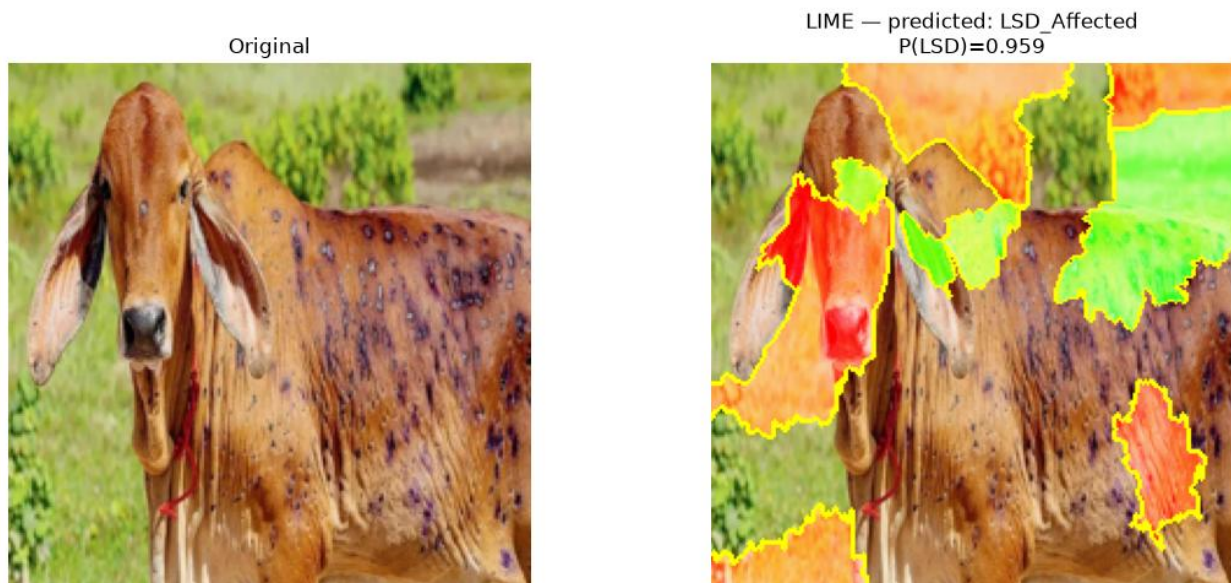


Figure 4.18. LIME explanation for a correctly classified LSD_Affected image

4.5.3 SHAP Observations

The SHAP visualization of the same LSD_Affected example shows both positive and negative spatial contributions. The stronger positive contributions are concentrated mainly on the cattle body, including regions where lesion patterns are visible, while other areas make weaker or negative contributions. The overlay therefore gives another view of the evidence supporting the predicted class and complements the regions emphasized by Grad-CAM and LIME.

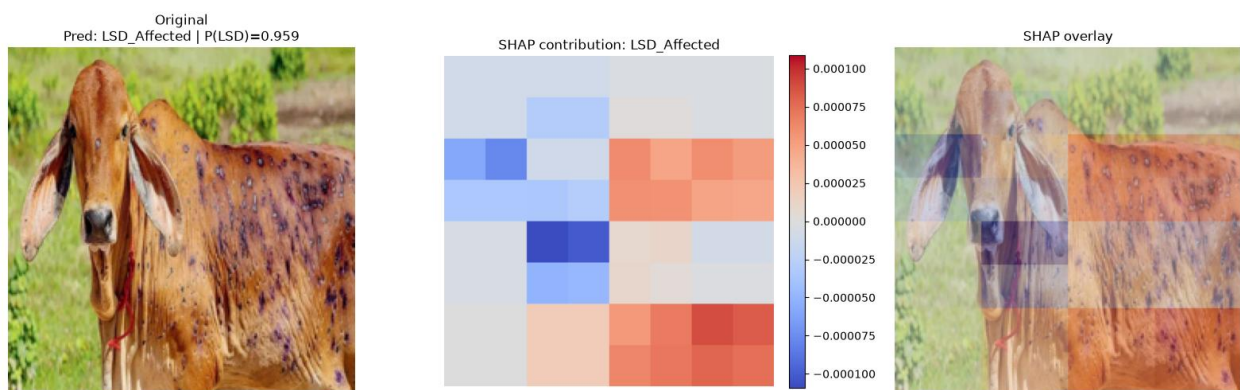


Figure 4.19. SHAP contribution map and overlay for a correctly classified LSD_Affected image

4.5.4 Combined XAI Discussion

The three explanation outputs are not identical, which is expected because they analyze the prediction in different ways. Even so, the supplied examples show a common tendency to emphasize regions on the cattle rather than only the surrounding background. In the affected examples, the highlighted areas frequently overlap with visible lesion-rich parts of the body. This agreement provides supportive evidence that the selected model is using visually relevant information when making these predictions. The explanations should still be treated as model-interpretation tools rather than as proof of clinical reasoning or veterinary diagnosis.

4.6 Overall Discussion

Taken together, the results show that all four transfer-learning models were able to learn the two-class classification task, but their strengths differed. ResNet50 produced the best F1-score and accuracy, MobileNetV2 offered the strongest AUC and lowest test loss, EfficientNetB0 produced the highest recall, and VGG16 produced the highest precision. This confirms that architecture choice changes the balance between sensitivity, precision, overall correctness, and computational cost.

The selection of ResNet50 is reasonable for this project because the chosen ranking criterion favors a balanced combination of precision and recall rather than maximizing only one measure. The XAI outputs further strengthen the analysis by showing that, in the examples examined, the model often places attention on cattle-body regions associated with visible skin abnormalities. The findings therefore support the usefulness of the proposed pipeline as a project-level image-classification and decision-support system, while the final interpretation should remain limited to the available dataset and experimental results.

CHAPTER 5

CONCLUSION

The project successfully demonstrates an image-based deep learning workflow for classifying cattle images into Healthy and LSD_Affected classes. Four transfer-learning models—EfficientNetB0, MobileNetV2, ResNet50, and VGG16—were trained and evaluated to select the best-performing model based on measured results.

The dataset contained 649 images and was checked for duplicate files, with no exact duplicates found. The images were organized into training, validation, and testing sets. Transfer learning and fine-tuning were applied to adapt the pretrained models to the LSD classification task.

Among the four models, ResNet50 achieved the best overall performance with 87.95% accuracy, 91.67% precision, 82.50% recall, 86.84% F1-score, and 95.06% AUC. MobileNetV2 achieved 86.75% accuracy, while EfficientNetB0 and VGG16 achieved 85.54% and 84.34% respectively.

The project also includes Explainable AI using Grad-CAM and LIME to visualize the image regions influencing predictions. Overall, the project achieves its goal of developing and evaluating an LSD image classification system with explainability. The results are promising for a project-level prototype, but it should be considered a research and decision-support tool rather than a replacement for professional veterinary diagnosis.

FUTURE WORK

Although the current system provides a complete prototype for LSD image classification and explanation, several improvements can be considered in future work.

First, the dataset size and diversity can be increased by collecting cattle images from different farms, breeds, lighting conditions, devices, disease stages, and backgrounds. This would improve generalization and reduce overfitting.

Second, the classification can be expanded beyond Healthy and LSD_Affected to include other skin conditions and visually similar cattle diseases. A multi-class system would make the application more useful in real-world situations.

Model evaluation can also be improved using cross-validation, repeated experiments, and detailed analysis of false positive and false negative cases. Different classification thresholds could also be investigated.

Explainability can be further strengthened by comparing Grad-CAM, LIME, and SHAP and validating the highlighted regions with veterinary experts. This would make the model's explanations more reliable.

Finally, the model could be integrated into a web or mobile application where users can upload cattle images and receive predictions, confidence scores, and explanation heatmaps. Testing with real-world images and expert feedback could help develop a more robust prototype for early LSD screening.